

Augmented Software Engineering @ Oneview Healthcare

The Good, The Bad & Where We Go From Here

Philip Brennan | ASE Meetup | 23 July 2026

Philip Brennan

Software Engineer (now SDM :)

Biography

- Computer Science graduate - finding it even more relevant these days!
- Operating systems and Hardware -> architecture, event driven systems
- Always want to be solving worthwhile problems ideally with elegant software.
- Mentoring and growing individuals and teams.
- AI to keep myself sane and in touch practically.
- Outside of software I'll be on my mountain bike!

The AI Engineering Landscape

Augmented Software Engineering

AI assists engineers in real-time. Human stays in control — guiding, correcting, deciding.

Dark Factories

Autonomous software delivery, from spec, code, tests, review and validation, deploy without continuous human input.

Many companies are in a scramble — building new SDLCs while new types of SDLC are being enabled and invented. Delivery doesn't go away - it is across old + new.

Augmented Engineering at Oneview

FrontMatter

- Health Tech - Patient Engagement and Clinical Collaboration
- Delivery includes software + hardware
- Complex integrations
- On premise -> Cloud

Augmented Engineering at Oneview

Evolution

- Github Copilot
- Prompts -> instructions -> skills -> agents
- Adoption / Mindsets
- AI Maturity Model

AUGMENTED · EXPERIENCE

Breadth of Application

Benefits the full SDLC, not just code. We have tackled several other bottlenecks.

Features · Tests · Documentation · Debugging · Determinism · Infrastructure

AUGMENTED · EXPERIENCE

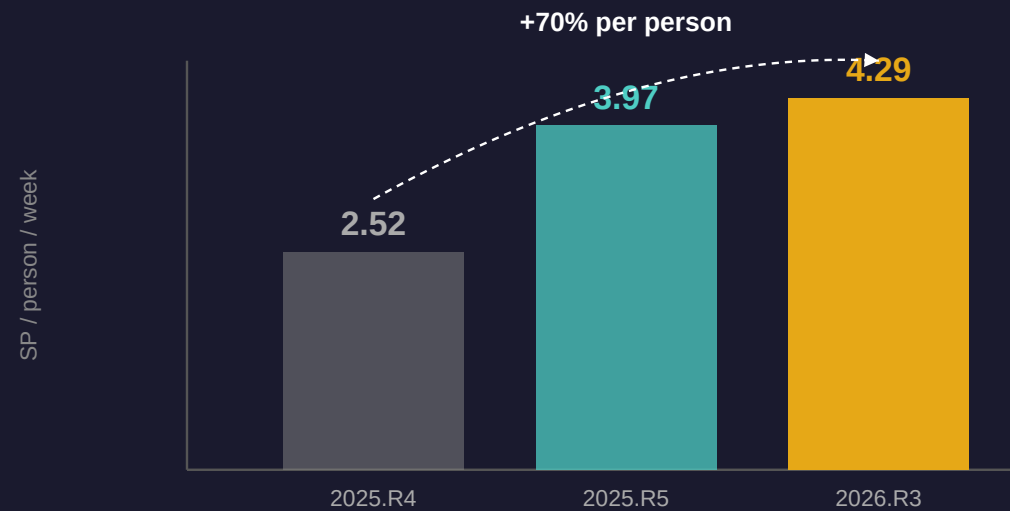
Augmented SDLC Speedups

Requirements & Specs	Technical review of PRD / FRD / stories Reverse-engineer specs & docs from legacy code (lift & shift)	HIGH 
Software Design	Attribute-driven design process distilled into skills	HIGH 
Build / Code	Feature development — 85% AI-authored · cross-discipline work	HIGH 
Review	1 human + 1 AI reviewer — each plays to strengths	MED-HIGH 
Test	Unified test generation across dev & test Harnesses simulating production conditions	HIGH 
Debug & Fix	Closed loop: debug → fix → verify · ~5x root-cause	HIGH 
Security Posture	SBOM critical/high vulns tending to zero (~50% cut in 2 months)	HIGH 

AUGMENTED · EXPERIENCE

Feature Development

1.7x delivered per person across full team.



Human owns architecture, acceptance criteria & review. AI accelerates implementation, boilerplate & test scaffolding.

AUGMENTED · EXPERIENCE

Bug Workflow

~5x Faster

Root-cause
investigation

Before

Read logs. Reproduce. Form hypotheses. Days of iterative digging.

~2x Faster

Full defect flow
(full cycle time)

Now

AI reads logs + ticket history + code. Debug, trace, propose root cause
Human validates the fix.

Full flow is measured & solid. Investigation gains are larger but harder to isolate.

AUGMENTED · EXPERIENCE

Bug Investigation — Before & After

Full defect flow

dev cycle time

Before

Now

~2x

Root-cause

investigation

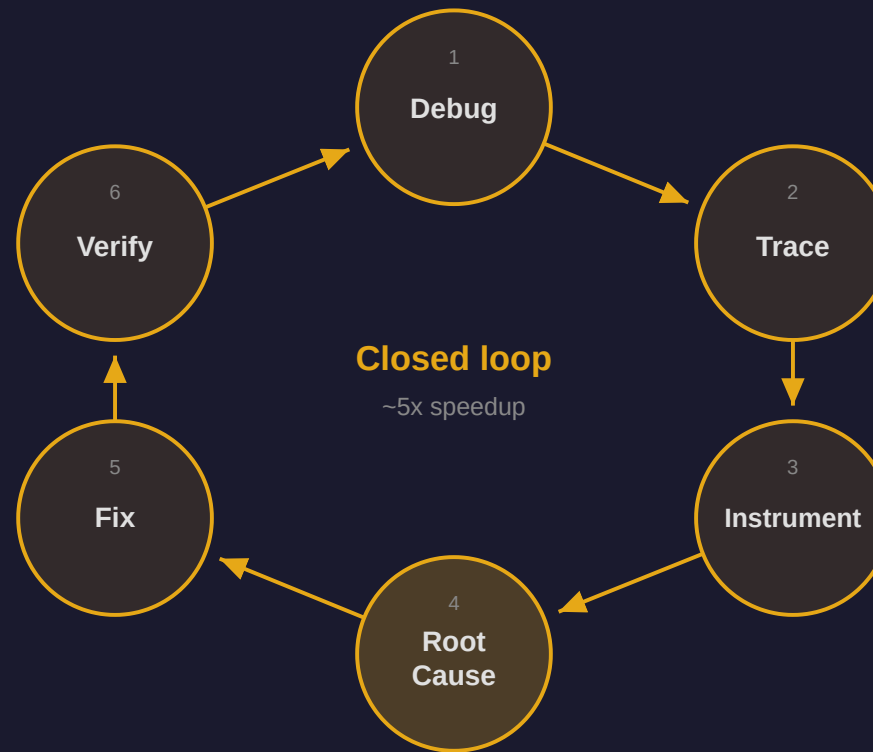
Before

Now

~5x

AUGMENTED · EXPERIENCE

Closed-Loop Debugging



AUGMENTED · EXPERIENCE

Beyond Code

Tackling Bottlenecks and Removing Excuses

- **Documentation** — generated from specs and code, not hand-authored
- **Test generation** — shared system, tests as executable intent
- **Investigation** — log analysis, history correlation, root cause
- **Determinism** — same shared skills, same quality bar, every time

Enforcement is key — determinism only holds if the skills are applied consistently.

AUGMENTED · EXPERIENCE

Code Generation

85%

of committed code
authored by AI

Still very high AI authorship — but with **easy human guidance** and course correction. The code can be reviewed, understood, and owned.

AUGMENTED · EXPERIENCE

How We Measure This

85% AI-authored — Measured over 2 months across engineering teams - human and AI analysis of PR's, interviewing with engineers.

~2x full defect flow — dev cycle time on comparable high-priority defects, same team, 2025 vs 2026. Typical dev time ~2.7 days → ~1.1 days. Measured by analysis of jira, codebase and deployment history.

~5x root-cause investigation — the diagnosis portion specifically. Larger gain, harder to isolate cleanly. Completely dependent on input quality. · Good logs which are correctly captured · Clear description/specification (may be edge cases / gaps)

Caveats: includes boilerplate and tests. Fix quality measured via regression rate — no degradation observed.

AUGMENTED · EXPERIENCE

T-Shaped Engineers

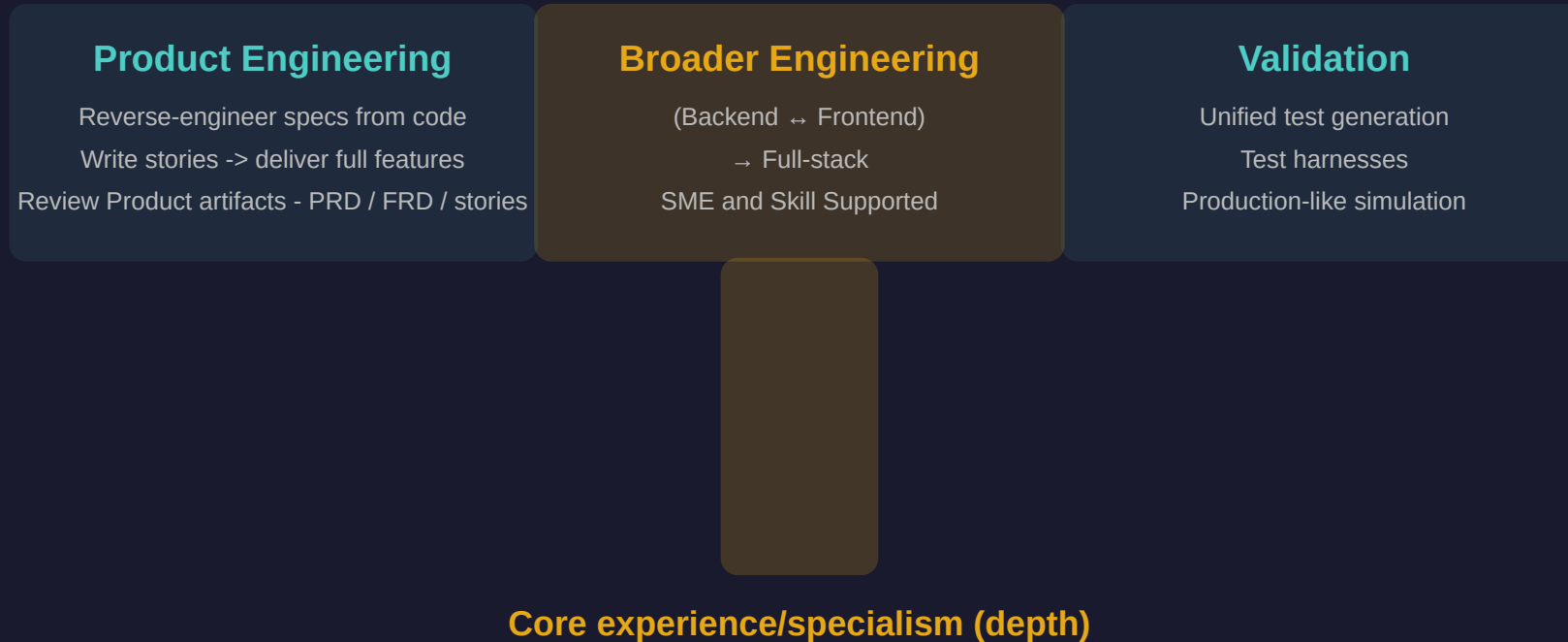
One deep skill, broadened in three directions — AI-enabled

- **Broader Engineering** — (Backend ↔ Frontend) → full-stack for BAU
- **Product Engineering** — reverse-engineer spec, research adjacent codebases
- **Validation** — unified test generation, production-like harnesses

AUGMENTED · EXPERIENCE

T-Shaped Engineers

Breadth — move left and/or right, AI-enabled



AUGMENTED · EXPERIENCE

T-Shaped — But Is It Real?

How we know it's genuine competency, not AI-masked shallowness:

- Code passes **same review standards** regardless of author's background
- Engineers can **debug and extend** what they shipped — not just generate it
- Cross-discipline PRs have **comparable defect rates** to specialist PRs

SMEs available for bringup and review - not support. Ownership excuses (I can't understand/own this) removed - AI can explain and help debug/document/test.

AUGMENTED · LEARNINGS

Skills (AI and Human) Drift

- Skills/instructions need to be kept in sync and be of highest quality
- Avoid conflicting/duplicate skills
- Not Invented Here syndrome can explode with AI. Parallel work/discovery.
- Engineers' core skills atrophying - if we mostly review code, how do we keep understanding it?

AUGMENTED · LEARNINGS

What We're Doing

Mitigations:

- 1 Human, 1 AI code reviewer - each focuses on their strengths.
- Review all outputs, surface the right decisions.
- Skills/instructions versioned and reviewed strictly, shared in plugin marketplace.
- Hooks to enforce use of skills.
- Mitigating hardware dependency

AUGMENTED · LEARNINGS

Solving the right problems?

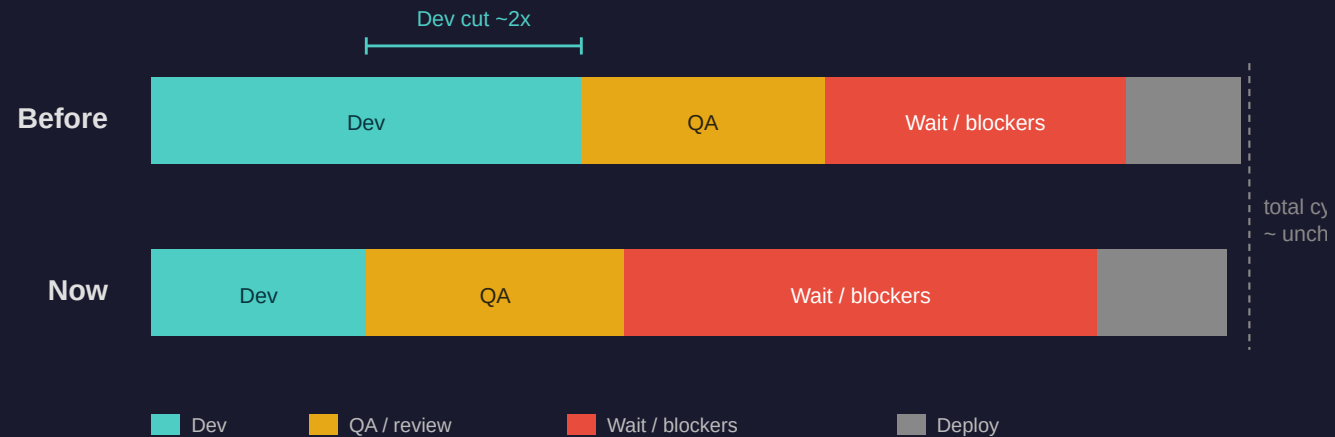
- Bottlenecks remain - Specification, Reviews, Validation, Customer integrations/configuration complexity
- Building the new plane while new types of planes are being invented.
- Delivering with old and new simultaneously.
- Constant expectations of improvement.
- Focus on outcomes

Tooling moves fast. Standardisation is a moving target.

AUGMENTED · LEARNINGS

Outcome Measurement

Dev got ~2x faster — but the **bottleneck just moved**.



Speeding up dev doesn't help if QA, blockers & hand-offs now dominate the cycle.

AUGMENTED · LEARNINGS

What We Should Have Measured

What we traditionally measured (outputs)

- PRs merged per sprint / Story points
- Cycle time (commit → deploy)

What actually matters (outcomes)

- Time-to-value (idea → customer use)
- Full cycle times including the remaining bottlenecks
- Defects to production

Dark Factories

Experience & Learnings

FACTORIES · EXPERIENCE

Consistency & Compliance

Always using the same skills

Eliminate human stylistic drift, fatigue, and shortcut-taking

Compliance with engineering principles is **built in**, not bolted on.

FACTORIES · EXPERIENCE

Breadth of Validations

Accessibility, performance, security — validations that humans routinely skip are now **automatically enforced**.

Speed on Less Novel Work

AgentFlow: an enterprise AI-agent orchestration platform (.NET 10)

- **Clean Architecture, four layers**
- **Built on open standards** — Microsoft Agent Framework (MAF), (MCP), A2A, Microsoft Entra Agent ID for identity.
- **Declarative, config-driven** — agents, workflows, MCP servers, and governance policies are defined in YAML and seeded into the database.
- **Workflow engine** — workflows are directed graphs of typed nodes with priority-ordered conditional edges.
- **Real identity per agent** — each agent authenticates as a Agent Identity, acting OBO an accountable human or as itself (app-only), with an append-only audit trail.

Capabilities

- **Multi-agent orchestration** — compose agents into pipelines and graphs; agents can delegate to each other as tools (delegation)
- **Three tool sources per agent** — built-in tool sets dynamically-discovered MCP server tools, and sub-agent tools.
- **Human-in-the-loop, two ways** — per-tool approvals for sensitive operations, and workflow Wait nodes for phase-gate reviews (with artifact review, file upload, context overrides, and free-text guidance).
- **Deterministic success/failure control**
- **Governance & safety** — policy-based tool-call enforcement, trust rings (0–4), prompt-injection detection, canary-token leak detection, rate limiting, and a kill switch that quarantines misbehaving agents.
- **Observability** — OpenTelemetry metrics, queryable execution history, audit events, and real-time SignalR notifications.
- **Model tiering** — Low/Standard/High deployments resolved per task

Comparison

- AgentFlow is a *governed control plane*; Gas Town is a *git-native coding-agent swarm*.
- They are complementary rather than competing. AgentFlow can orchestrate a coding agent (or even a Gas Town run)
- AgentFlow's differentiators are: **governance, per-agent enterprise identity with human accountability (OBO) explicit workflow control** with success/failure determination and HITL gates.
- Gas Town's differentiators are **git-native persistence, merging, massive coding-agent parallelism, and attribution-driven model comparison**.

FACTORIES · EXPERIENCE

Ideal for longer-running, well-defined work.

Refactors · Migrations · Cookie cutter features

Governance - policies for tool use and patterns are enforced.

Once working, same quality every time without human variance

FACTORIES · LEARNINGS

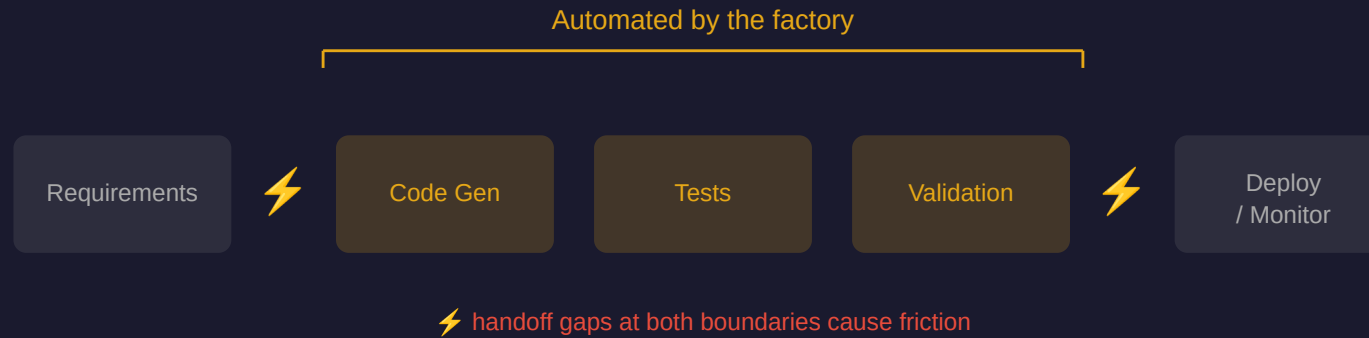
Rome wasn't built in a day

- Building the factory is a massive investment
- We've built a general purpose harness rather than purely focusing on SDLC - good + bad
- Models did not include Anthropic - less forgiving on inputs
- Human translation of designs in augmented was masking design system issues.
- Limited availability bottlenecked on SME's - now solved

FACTORIES · LEARNINGS

Factory Coverage Gap

We automated the **middle** of the pipeline — not the ends.



If incomplete outputs handed to engineers to fix factory output instead of building from scratch - this dilutes all advantages and decreases buy-in.

FACTORIES · LEARNINGS

Token Costs

Have been corrected to economic reality Still a risk Target less challenging problems than front end development

FACTORIES · LEARNINGS

Human in the Loop

Has **huge value** and is not just box-ticking.

Input quality is even more critical. On new features this can vary. Checking intermediate outputs matters.

The Wider Problems

Industry observations

THE WIDER PROBLEMS

Top-Down vs Grassroots

- Company AI objectives don't always align with what engineers actually need on the ground.
- Tangible objectives that can be aligned to.
- Metrics and clear outcomes
- Buyin is key

ACT 3

Where We Go From Here

- Parallel Augmented and Dark Factory development
- Coalesce context / approaches where possible
- Continue working on non coding bottlenecks
- Retarget our Agent Platform use cases
- Spec driven Development

ACT 3

Where We Go From Here

- What human factors are slowing us down? (Not just shifting to AI first mindset)
- How do we mitigate these?
- Organizational structure changes

NEXT STEPS

The Pragmatic Model: AI-Reviewed ASE

Augmented Software Engineering — **AI-reviewed & AI-debugged** — is the pragmatic model for most companies today.

- Bridges manual development ↔ autonomous dark factories
- Humans stay in the driver's seat for **design & architecture**
- AI speeds the bottlenecks — **code review & debugging**
- Human-in-the-loop for value, not box-checking

Human-written code + multi-agent AI verification gates = a resilient development loop.

NEXT STEPS

Matching the Factory's Consistency

The Dark Factory's consistency edge can be **practically equalled** in ASE.

- Factories win on consistency by removing human drift, fatigue & shortcuts
- Enforce approved, **deterministic "skills"** in the workflow
- Treat AI agents as a registry of **executable, bounded micro-services** — not creative chat companions
- Same mathematical consistency, injected into human-driven development

NEXT STEPS

Measure Outcomes, Not Outputs

Shift from PRs and LOC to **quality** and **time-to-value**.

NEXT STEPS

Invest in Principles

Real engineering principles are the foundation.

AI **amplifies** them — doesn't replace them.

Thank You

Questions & Discussion

Philip Brennan | ASE Meetup | 23 July 2026